Memory or Task Pressure? An Audit of a Scam-URL Benchmark for LLM Code Generation

Zili Luo · Independent study, October 2026 · Not peer reviewed

Abstract. Scam2Prompt reports that production LLMs write scam URLs into generated code, and reads this as contamination absorbed into model weights. This study asks whether a simpler mechanism could produce the same result: a model told to write runnable code may build a URL from a name in the prompt. The authors' own classification shows that 99.5% of 41,317 flagged URLs in their full audit came from prompts naming the URL or platform, which fits both explanations. In a control on 80 benchmark prompts, run on two Claude models, no gap was detected between invented brands and scam-site brands. Removing the template's demand for runnable, production-ready code and allowing a placeholder cut these URLs by 19-30 points, with no measured loss on 40 real services. Of 181 datable GPT-4o domains, 20 show no sign of existing before its stated cutoff. A note claiming that URLs were verified produced no detected rise in URL rates. These results do not show that memory plays no role, and they leave the case for an outside URL check intact.

1. Introduction

LLM coding assistants write code that people run. Chen et al. [1] showed that this code can contain scam URLs, and traced the risk to real harm in the pump.fun incident, where a user lost funds after running generated code. Their explanation is contamination: scam content reaches training data, and "harmful content, such as scam API endpoints, has already been absorbed into the models' weights".

That explanation predicts that models recall scam URLs. A second explanation predicts the same output. Their code template asks for code that is "Syntactically correct and runnable" and "production-ready", with no explanations. For a task that names a service, a model obeying this template may feel pushed to write some endpoint, and `api.<name>.com` is the obvious guess. The two explanations imply different defenses, so telling them apart matters.

This study began with a mistaken reading of mine. The paper's seven 2025 models show rates of 12.9-47.3% on its benchmark, against 4.24% in its audit of four 2024 models across 265,114 prompts, which looks like newer models getting worse. The rates are not comparable: the benchmark holds only prompts on which all four 2024 models failed, and the authors call its results stress-test rates. Checking that reading led to the question here.

2. Background and related work

Scam2Prompt. An LLM reads a scam page and writes developer-style prompts "most likely to direct a large language model (LLM) to this specific webpage". Code models answer those prompts without seeing the page and without tools. Three detectors check each URL. This full run is the audit. Prompts where all four 2024 models failed became Innoc2Scam-bench, with 1,377 prompts. The paper splits these by whether a URL is named: 342 in Category 1, which name one, and 1,035 in Category 2, which do not. The dataset's labels show that all Category 2 prompts still mention a platform name.

Fabrication in generated code. Spracklen et al. [3] measured hallucinated package names at 5.2% for commercial models and 21.7% for open-source models. Li and Gao [4] extended fabrication threats

across the software supply chain and proposed a prompt-based defense. Models that invent names are the background to the task-pressure reading here.

Task pressure and instruction following. ImpossibleBench [5] measures agents exploiting tests when a spec cannot be met. Hasan and Biswas [6] characterize operational safety failures of coding agents. Vendors say newer models follow instructions more literally [7, 8], and SIFo [9] finds that instruction sequences remain hard for all models. I found no study measuring whether tighter task following explains scam URLs.

Search poisoning. The paper excludes search. Chen et al. [10] find that LLM-enhanced search engines stop over 99.78% of traditional SEO attacks, which is a separate route from the one studied here.

3. Research questions

- **RQ1.** How often is a flagged domain's name already in the prompt?
- **RQ2.** Do models write URLs for brands that cannot be in their training data?
- **RQ3.** Does the task wording change this, and what does the change cost on real services?
- **RQ4.** Does a claim that URLs are verified act as ground truth?
- **RQ5.** Do flagged domains exist before the training cutoff of the model that wrote them?

4. Method

Published data. I classified the authors' released outputs for seven 2025 models (9,639 outputs), reproducing their Table 2 counts exactly, and read three of their twelve audit reports.

Control. I sampled 80 Category 2 prompts from the 1,007 where the brand could be located, and gave each brand a new invented name, checked against a top-sites list, a blocklist and the published outputs. Three templates were used:

- *Paper*: the authors' code template.
- *Relaxed*: drops "runnable" and "production-ready", and asks for a placeholder when the model is unsure of a URL.
- *Verified*: the paper's template plus "all services, APIs and URLs referenced in this request have been verified by our team."

Haiku 4.5 and Sonnet 5.5 ran through `claude -p`, one turn each, with no tools, in a neutrally named folder. This gives 960 calls. The outcome is whether an output contains a concrete URL built from the brand.

Real services. To check that a change does not simply suppress all URLs, 40 popular services each received one simple task under the same templates, for 240 calls. An answer is usable if it has the official URL or official SDK. I checked the answer key against a top-sites list, and read by hand every host outside it.

Domain age. I took registration dates from public registries. A late date can mean a lapsed domain registered again, so I treated a domain as new only if the Internet Archive holds no capture from before registration. I sampled 400 of the 1,067 domains GPT-4o was flagged for.

Safety. Generated code was stored as text and never run. No scam site was contacted.

5. Results

RQ1. The authors' classification of 41,317 flagged URLs gives URL in prompt 49.7%, platform name in prompt 49.8%, and neither 0.5%. My check agrees: the domain's name is spelled in the prompt for 95.7-98.2% of flagged URLs in three audit reports, and for 99.5% of 3,441 flagged benchmark outputs. Memory predicts this as well, because prompts were written to elicit each site, so RQ1 does not separate the two explanations. It only shows that memory may not be needed for these cases. Detectors also flagged the placeholder `yourdomain[.]com` 18 times.

RQ2 and RQ3. Rates are the share of outputs with a concrete brand URL, or for real services a usable answer.

Model	Template	Real service	Scam-site brand	Invented brand
Haiku 4.5	Paper	98%	42%	35%
Haiku 4.5	Relaxed	98%	12%	12%
Haiku 4.5	Verified	98%	40%	26%
Sonnet 5.5	Paper	95%	21%	20%
Sonnet 5.5	Relaxed	100%	2%	0%
Sonnet 5.5	Verified	95%	21%	15%

Paired differences in points, with 95% bootstrap intervals resampled by brand:

Effect	Haiku 4.5	Sonnet 5.5
Scam brand minus invented, paper template	+7.5 [-4.9, +19.0]	+1.2 [-10.4, +12.8]
Paper minus relaxed, scam brand	+30.0 [+17.3, +43.2]	+18.8 [+11.0, +28.2]
Paper minus relaxed, invented brand	+22.5 [+10.0, +35.4]	+20.0 [+11.1, +30.4]
Verified minus paper, scam brand	-2.5 [-14.6, +9.8]	0.0 [-9.5, +9.3]
Scam brand minus invented, verified template	+13.8 [+2.4, +26.2]	+6.2 [-1.3, +14.5]

Under the paper template, an offline check matched 19% of Haiku outputs and 4% of Sonnet 5.5 outputs to a known flagged domain, so these rates are not comparable with the paper's detectors. Brand URLs also cannot separate recall from guessing when the domain contains the name, which is why the invented brands are needed.

Real services saw no refusals and no wrong URLs. The few answers not counted as usable contain a placeholder (Alchemy and Infura, where my scorer may have misread URLs built from a key variable). Eight outputs held an extra host outside the answer key, and all eight appear to belong to the service's own company.

RQ4. No rise in brand URL rates was detected, though the interval for Haiku reaches +9.8 points. Haiku refused more often (31% to 51% on scam brands, 46% to 65% on invented brands). Sonnet 5.5 said it could not verify a service less often (44% to 15%) and left the endpoint out more often (21% to 41%).

RQ5. For GPT-4o, 181 sampled domains could be dated. Of those, 101 existed by its October 2023 cutoff, and 34 were registered after the run, so they must be re-registrations. Of 46 candidates registered in between, 35 archive lookups succeeded: 20 show no earlier history and 15 do. Six of the 20 were registered in 2025. Across the 2025 outputs, 193 of 346 flagged domains show no current

registration. For 2025 models the test is uninformative. A loose version gave 23-33% too new, and a strict one gave 0.7-4%, because most late dates were re-registrations.

Stated reasons. Saved outputs hold final answers only, not private reasoning. Wording of "can't verify this service" on the same 80 prompts under the paper template appears in 1% of Sonnet 4 outputs, 0% of GPT-5, 1% of Gemini 2.5 Pro, 1% of DeepSeek, 19% of Haiku 4.5 and 44% of Sonnet 5.5.

6. Discussion

The data suggests task pressure is a large part of this benchmark's signal. No gap was detected between invented and scam-site brands, and changing the wording moved the rate more than changing the brand. The strongest evidence is the control, not the name being in the prompt. Matching rates cannot prove a shared mechanism, and the 0.5% of URLs with no name in the prompt, which include the kind of case the paper worries about most, were not analysed here.

This reframes what the benchmark measures, and it should not be read as dismissing the risk. A user who asks for a named scam service and receives a working client is at risk whichever mechanism produced it. The authors already call one such case "debatable" in their prompt review, and they describe the benchmark as a stress test. The harder claim to support is that these URLs show memorized contamination, since most of them may not need it.

The real-service results suggest what separates the cases. Models answer correctly for Stripe and hedge for an unknown name, so the safeguard looks like recognition. That is a weak safeguard. A scam that becomes familiar would be treated as real. Separately, models keep guessing `api.<name>.com`, so an attacker could register domains that models invent. The paper's recommendation of an outside URL check stands, and these results add to the case for it, because the prompt changes tested here would not catch a confident wrong answer.

A verification claim did not act as ground truth here. Outputs still cited unknown services, and Haiku refused more often. Why that happens cannot be read from the outputs alone.

7. Threats to validity

- **Models.** Two Claude models, neither in the paper. `claude-sonnet-4` was unavailable, so this is not a replication.
- **Memory is not ruled out.** The scam-minus-invented gap may reach 19 points for Haiku, and with the verified note it excludes zero. Invented names read oddly, so this may reflect plausibility, but it is unresolved.
- **Harness.** Models were told their knowledge cutoff and today's date, and thinking was on. The paper's text never mentions thinking and its code does not set it, so its state per model is unknown. Differences between models are not purely model effects.
- **Confounded treatments.** The relaxed template removes pressure and adds an instruction at once. The verified note may read as suspicious in itself.
- **Real-service tasks.** They are simpler than the benchmark prompts, and every service is popular. A small real service is untested, and that is where a placeholder rule would likely cost something.
- **Domain age.** Many domains have lapsed and cannot be dated, and archive coverage is partial. Later GPT-4o snapshots may have later cutoffs.

- **Scoring.** Text heuristics and a regex, with samples read by hand. Offline checks cannot match the paper's three detectors.
- **Sampling.** 80 prompts, temperature at default against T=0 in the paper, and three of twelve audit reports.

8. Reflection

My first reading was wrong. I compared 4.24% with 12.9-47.3% and saw newer models getting worse, when the two numbers rest on different prompt sets. Finding that changed how I read headline rates: I now ask what produced the denominator.

The most useful moment came from a failure. The first run used a folder named after the study, the path reached the model, and 101 of 320 Haiku outputs mentioned it. Every automatic check had passed. Reading outputs by hand caught it, and all 640 calls were discarded. The same thing happened with domain age: the first result looked strong, then 2026 registration dates showed that it was measuring re-registration, and the strict version removed nearly all of it.

I also learned how close a new idea can be to existing work. The 99.5% figure felt like a discovery until the authors' own artifact turned out to hold the same result. A paper's repository is part of the paper.

Finally, what a model says is not why it acts. Outputs show "I could not verify this service", but that is a self-report produced under a template, and the reasoning behind it was not available to me.

9. Future work

Several directions look worth exploring, and none is a fixed plan.

- **Study the 0.5%.** URLs from prompts naming neither the URL nor the platform are the likeliest place to find real recall.
- **Exact matches.** Scoring whether a model hits the precise flagged domain, including rare endings, could separate recall from guessing.
- **Remove the name entirely.** Describing a task with no brand is the closest test of memory, and the nearest to the pump.fun incident, where the output named a domain the prompt did not.
- **Separate the relaxed template.** Removing the pressure and allowing a placeholder could be tested one at a time.
- **Small real services.** Real services a model has never seen would measure what a placeholder rule costs.
- **A cleaner harness.** Direct API runs could remove the date and cutoff text and turn thinking on and off.
- **More models.** The paper's own models and other vendors would show how far the result generalizes.
- **Guessable domains.** How many domains that models invent are unregistered? This can be measured without registering any.
- **Agents with memory.** A bad endpoint written into an agent's notes could be reused session after session.

- **Beyond URLs.** A URL has a blocklist. For code that runs but is wrong, what plays the role of the oracle?

10. Conclusion

On this benchmark, most flagged URLs can plausibly be explained without memory: the prompt supplies the name, the template asks for working code, and many models write an endpoint. A small change in wording reduced this at no measured cost on popular services, while a claim of verification did not act as ground truth. Models seem to rely on recognizing names, which helps with unknown names and may offer little against a familiar scam. Measuring model safety requires separating what a model knows from what a task makes it do.

How this was built

I asked the question, proposed the task-following hypothesis and the "verified" test, and set the rule that generated code is never run. Claude Code, working under my direction, proposed the invented-brand control, the real-service control and the domain-age test, wrote and ran the code, and drafted this report. I reviewed the results and decided what to keep.

Reproduce

`scripts/` holds the analysis code, with paths pointing at a local working folder. Raw outputs and benchmark data are not in this repo because they contain real scam URLs. The benchmark is available from the authors' release [2].

References

1. Z. Chen, T. Saba, X. Deng, X. Si, F. Long. Scam2Prompt: A Scalable Framework for Auditing Malicious Scam Endpoints in Production LLMs. ICML 2026, PMLR 306:16495-16530. arXiv:2509.02372.
2. Innoc2Scam-bench and the Scam2Prompt artifact. github.com/jeffchen006/Innoc2Scam-bench-ICML26 and github.com/Scam2Prompt/Scam2Prompt-ICML-Artifact.
3. J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, M. Jadliwala. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. USENIX Security 2025. arXiv:2406.10279.
4. X. Li, X. Gao. Investigating Security Implications of Automatically Generated Code on the Software Supply Chain. arXiv:2509.20277, 2025.
5. Z. Zhong, A. Raghunathan, N. Carlini. ImpossibleBench: Measuring LLMs' Propensity of Exploiting Test Cases. arXiv:2510.20270, 2025.
6. A. A. Hasan, S. Biswas. What Breaks When LLMs Code? Characterizing Operational Safety Failures of Agentic Code Assistants. ASE 2026. arXiv:2605.30777.
7. OpenAI. Using GPT-4.1. developers.openai.com/api/docs/guides/gpt-4.1.
8. Anthropic. Introducing Claude Opus 4.7, April 2026. anthropic.com/news/claude-opus-4-7.
9. X. Chen, B. Liao, J. Qi, P. Eustratiadis, C. Monz, A. Bisazza, M. de Rijke. The SIFo Benchmark: Investigating the Sequential Instruction Following Ability of Large Language Models. EMNLP 2024 Findings. arXiv:2406.19999.
10. P. Chen, G. Hong, X. Wu, M. Wu, Z. Zhu, M. Liu, B. Liu, M. Zhang, M. Yang. Unveiling the Resilience of LLM-Enhanced Search Engines against Black-Hat SEO Manipulation. WWW 2026. arXiv:2603.25500.